DCCP: Datagram Congestion Control Protocol

Submitted by

Rathul Hasan Saion

ID: IT24037

Supervised by

Dr. Nazrul Islam

Associate Professor

Department of ICT, MBSTU

Department of Information and Communication Technology

Mawlana Bhashani Science and Technology University (MBSTU), Tangail, Bangladesh

Presentation Outline

01 Why do we need DCCP?

02 What is DCCP?

03 DCCP in the protocol stack

04 Connection setup and header size

05 Congestion control and key features

06 Security, applications and conclusion

DCCP

Unreliable • Congestion-controlled

Transport protocol for real-time data, running directly over IP

Why Do We Need DCCP?

TCP is too slow for real-time data; UDP is careless about the network.

1

TCP delays real-time data

Retransmission and in-order delivery make late packets stall live audio, video and games.

2

UDP has no congestion control

Unregulated UDP flows can overload links and starve other traffic, risking congestion collapse.

3

Every app reinvents the wheel

Without a standard, each application builds its own rate control, often poorly or unfairly.

4

A middle path is needed

Real-time apps want unreliable delivery that still shares the network fairly: this is DCCP.

What Is DCCP?

Unreliable, congestion-controlled datagram transport

- Standardised by the IETF in RFC 4340 (March 2006)
- Connection-oriented, with message-based (datagram) delivery
- Adds congestion control but no retransmission
- Runs directly over IP (protocol 33) or inside UDP

2005

First DCCP implementation in Linux 2.6.14

March 2006

RFC 4340 published, with CCID 2 and CCID 3

Nov 2012

RFC 6773 defines UDP encapsulation

July 2025

Linux 6.16 removes DCCP support

DCCP in the Protocol Stack

Without DCCP: UDP + app-level control

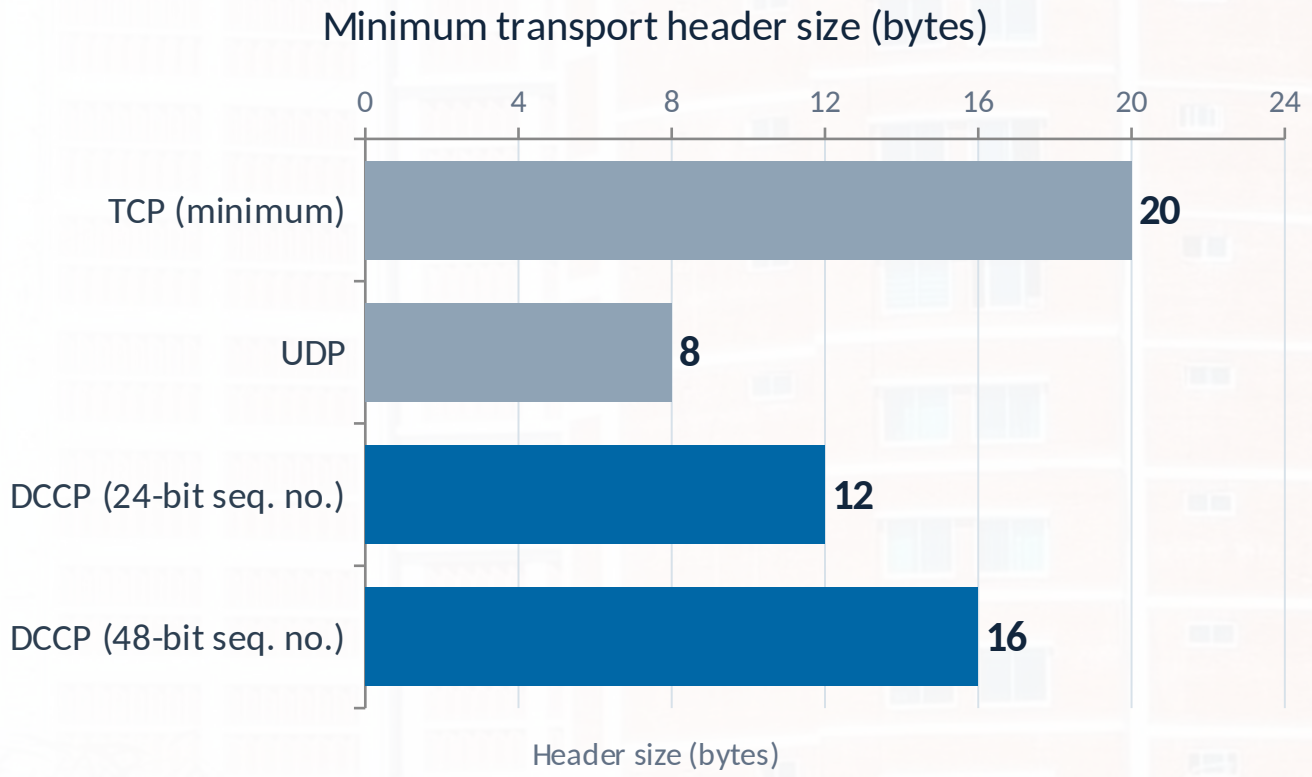
- Application (media / game)
- RTP / custom protocol
- **Rate control inside each app**
- **UDP**
- IPv4 / IPv6

With DCCP: transport-level control

- Application (media / game)
- RTP (optional)
- **DCCP (connection + CCID congestion control)**
- UDP encapsulation (optional)
- IPv4 / IPv6

DCCP adds congestion control directly above IP, without reliability

Connection Setup and Header Size



3-way handshake

Request, Response and Ack open a connection; Init Cookies protect the server.

10 packet types

Request, Response, Data, Ack, DataAck, CloseReq, Close, Reset, Sync, SyncAck.

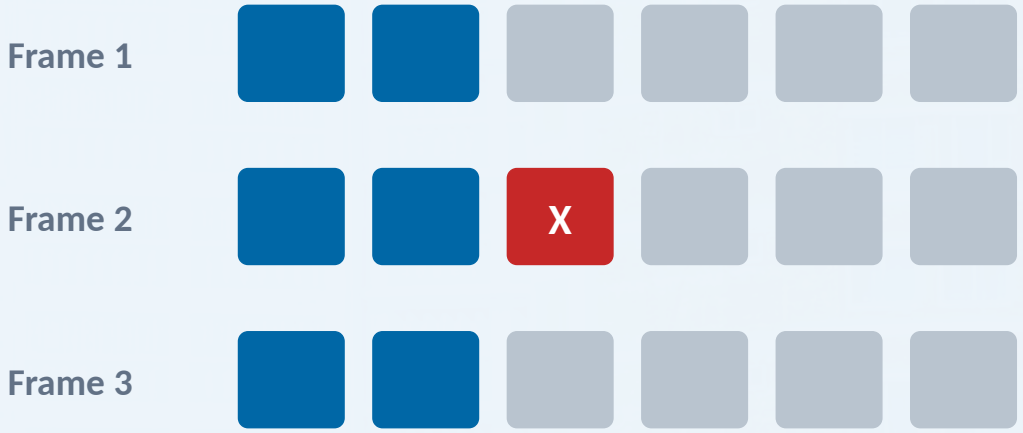
Why it matters

A small header with packet-based sequence numbers keeps overhead low for real-time data.

DCCP: Datagram Congestion Control Protocol

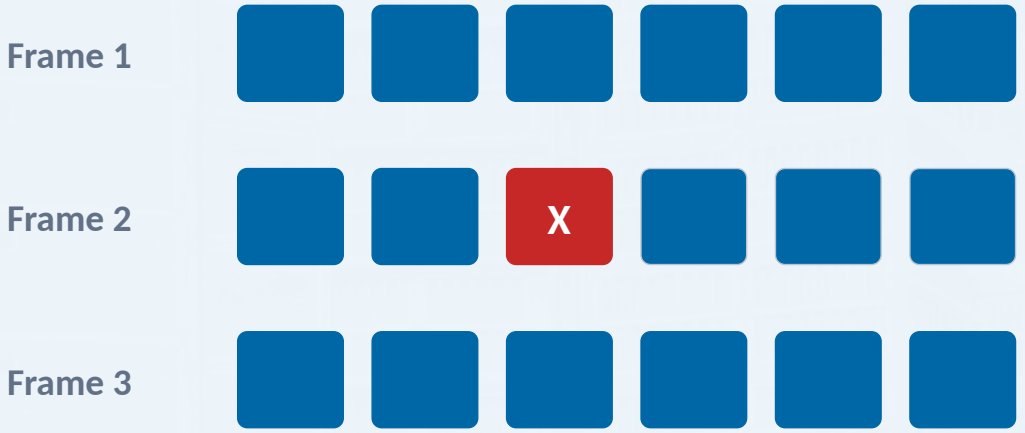
Unreliable Delivery: No Head-of-Line Delay

TCP: reliable, in-order delivery



One lost packet delays every frame behind it.

DCCP: unreliable, congestion-controlled

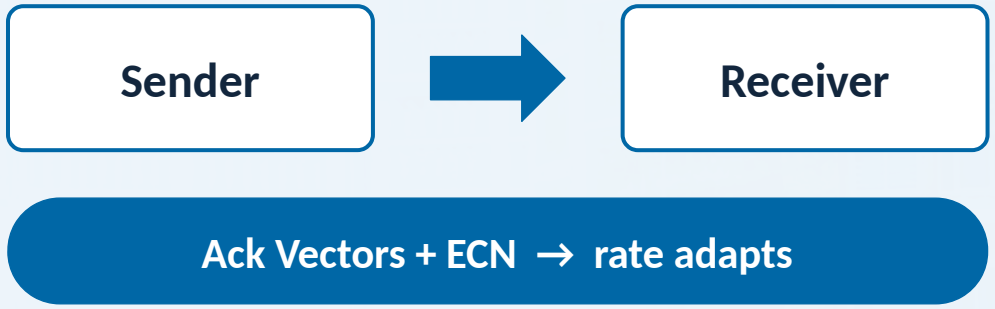


A lost packet is skipped; later data still arrives on time.

Delivered Lost packet Waiting

Congestion Control and Key Features

Congestion control IDs (CCIDs)



CCID 2 TCP-like AIMD: bursty, uses all spare bandwidth (RFC 4341).

CCID 3 TFRC: smooth sending rate for streaming and VoIP (RFC 4342).

Other key features

- Feature negotiation lets endpoints agree on the CCID and options
- Ack Vectors report which packets arrived, even without retransmission
- Partial checksums let apps accept slightly corrupted audio or video
- 32-bit Service Codes identify the application using a port

Security Considerations

1

Sequence validation

Packet-based sequence numbers and Sync packets help reject forged packets.

2

Init cookies

A server can answer a Request statelessly with an Init Cookie, resisting connection floods.

3

Encryption is external

Confidentiality is left to IPsec or DTLS running over DCCP.

Trade-off: DCCP has no built-in encryption, so sensitive traffic needs IPsec or DTLS (RFC 5238).

Applications and Challenges

Applications

Streaming — Live audio and video, where timeliness beats reliability

VoIP — Internet telephony and video calls (RTP over DCCP, RFC 5762)

Gaming — Online multiplayer games that need low latency

Challenges

- NATs and firewalls often drop IP protocol 33, so UDP encapsulation is needed
- No support in Windows, macOS or browsers; Linux removed it in 6.16 (2025)
- Developers kept using UDP with custom control, and later QUIC
- Choosing and tuning a CCID adds complexity

DCCP: Datagram Congestion Control Protocol

Conclusion and References

01 DCCP is an unreliable, congestion-controlled transport for datagrams

02 Pluggable CCIDs: TCP-like (CCID 2) and smooth TFRC (CCID 3)

03 No retransmission delay, yet fair to other traffic

04 Technically sound, but adoption stayed limited by NAT and platform support

Selected references

[1] E. Kohler, M. Handley and S. Floyd, “Datagram Congestion Control Protocol (DCCP),” RFC 4340, IETF, 2006.

[2] S. Floyd and E. Kohler, “Profile for DCCP CCID 2: TCP-like Congestion Control,” RFC 4341, IETF, 2006.

[3] S. Floyd, E. Kohler and J. Padhye, “Profile for DCCP CCID 3: TFRC Congestion Control,” RFC 4342, IETF, 2006.

[4] T. Phelan, G. Fairhurst and C. Perkins, “DCCP-UDP: A DCCP UDP Encapsulation for NAT Traversal,” RFC 6773, IETF, 2012.

[5] E. Kohler, M. Handley and S. Floyd, “Designing DCCP: Congestion Control Without Reliability,” ACM SIGCOMM, 2006.

Thank you

Questions and Discussion

Rathul Hasan Saion

ID: IT24037 | Department of ICT, MBSTU

Supervised by Dr. Nazrul Islam, Associate Professor, ICT, MBSTU